



Agent Boundary, Tool & Data Access Enforcement Engine

End-to-End Engineering Implementation Specification

Developer-ready design for Frontend, FastAPI Backend, PostgreSQL, Runtime Interception, Security, Audit, QA and Deployment

Field	Details
Purpose	Define how GuardianIQ constrains AI agents, models, tools/APIs and enterprise data at runtime so unauthorized or policy-violating actions are stopped before execution or data disclosure.
Builds on	GuardianIQ Relationship Architecture; Step 4 Governance Event Architecture; Policy Binding, Resolution & Runtime Enforcement Engine specification.
Primary stack	React, FastAPI/Python, PostgreSQL, GuardianIQ policy engine, event/outbox architecture.
Core outcome	Agent identity + autonomy + model + tool + data + policy + approval are evaluated as one server-side enforcement chain before any controlled action executes.
Document status	Engineering implementation specification / development handoff.

Engineering principle: An agent being 'registered' or a tool/data source being 'attached' is not enforcement. Enforcement exists only when the server-side execution path validates the boundary immediately before invocation and blocks, transforms, routes for approval, or allows the exact request.

Contents

1. Executive Summary
2. Scope and Non-Goals
3. Relationship to Existing GuardianIQ Architecture
4. Target Enforcement Architecture
5. Governed Runtime Request Model
6. Agent Identity and Runtime Principal
7. Agent Autonomy Boundary Model
8. Model Invocation Boundary
9. Tool Registry and Capability Model
10. Tool Permission Model
11. Tool Execution Gateway
12. Data Source Registry and Classification
13. Data Permission Model
14. Data Access Enforcement and Transformation
15. Cross-Boundary Decision Model
16. Relationship Resolution
17. Policy Engine Integration
18. Approval, Delegation and Exception Integration
19. Runtime Authorization Token and TOCTOU Protection
20. Database Physical Design
21. Backend Module Architecture
22. Backend Service Specifications
23. Runtime Interceptor Specifications
24. REST API Contracts
25. Frontend Information Architecture
26. Frontend Screen Specifications
27. Frontend Component Behavior
28. Governance Events and Audit Evidence
29. Security Architecture

30. Secrets and Credential Handling

31. Performance, Caching and Resilience

32. Error Handling and Fail-Closed Rules

33. Observability and Operations

34. Detailed End-to-End Scenarios

35. Testing Strategy

36. Acceptance Test Catalogue

37. Implementation Sequence

38. Resource/Workstream Breakdown

39. Deployment and Migration

40. Definition of Done

41. Handover Checklist

Appendix A. Core Enums

Appendix B. Example Permission Records

Appendix C. Suggested Code Structure

1. Executive Summary

This subsystem is the GuardianIQ enforcement plane for AI execution. It answers four runtime questions before a governed request is permitted: Who/what is acting? What is the agent allowed to do? Which model/tool/data is being requested? Do relationships, permissions, policies and approvals authorize this exact request?

Canonical flow: AGENT REQUEST → AUTHENTICATE RUNTIME PRINCIPAL → LOAD AGENT BOUNDARY → VALIDATE MODEL → VALIDATE TOOL CAPABILITY → VALIDATE DATA ACCESS → RESOLVE RELATIONSHIPS → RESOLVE/EVALUATE POLICIES → APPROVAL/EXCEPTION CHECK → ISSUE SHORT-LIVED AUTHORIZATION → EXECUTE THROUGH CONTROLLED ADAPTER → RECORD EVENTS/EVIDENCE

- Agent boundaries define maximum autonomy, operations, tools, models, data classes, monetary/volume limits and execution environments.
- Tool permissions are operation-level, not merely tool-level; READ, WRITE and EXECUTE are independently governed.
- Data permissions control source, classification, purpose, fields, row scope, masking/tokenization, export and model/provider compatibility.
- Policy enforcement is layered on top of explicit relationships and permissions; policy ALLOW cannot invent a missing tool/data entitlement.
- High-risk execution fails closed if required control context cannot be established.
- Every decision is reconstructable using request_id, correlation_id, agent/model/tool/data identifiers, policy versions, approval and execution outcome.

2. Scope and Non-Goals

In Scope	Not the primary responsibility of this engine
Agent runtime identity and autonomy boundary	Training/fine-tuning models

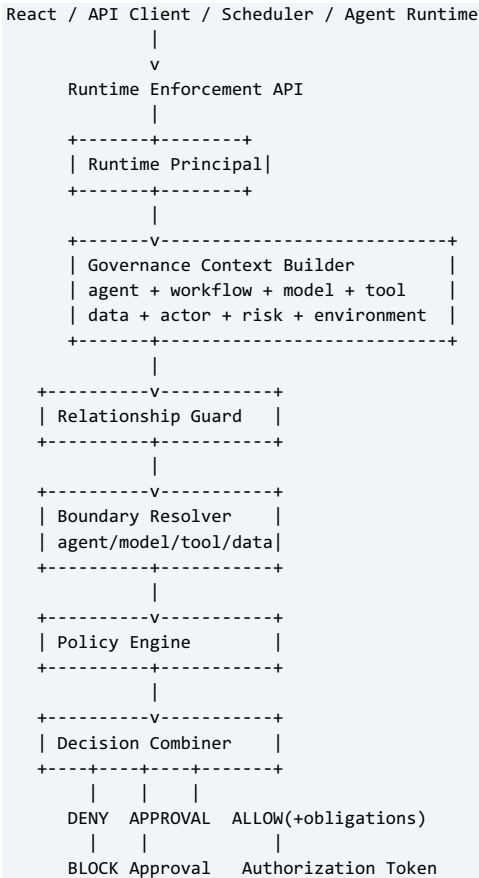
Model/provider invocation authorization	Full enterprise IAM/PAM replacement
Tool capability and operation authorization	Implementing every third-party tool connector
Data-source and field-level governance decisions	Replacing database-native security/RLS where it already exists
Pre-execution interception and enforcement	General-purpose network firewall
Policy/approval/exception integration	Natural-language policy generation as authoritative logic
Runtime evidence and audit events	Secret vault implementation itself

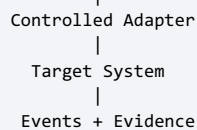
3. Relationship to Existing GuardianIQ Architecture

Existing concept	How this specification uses it
USES_MODEL	Validates that agent/workflow may invoke a model and under which constraints.
USES_TOOL	Validates tool relationship and access_mode before invocation.
USES_DATA_SOURCE	Validates data source access and sensitivity constraints.
PARTICIPATES_IN	Confirms agent is authorized in the workflow context.
GOVERNED_BY	Supplies applicable policy bindings.
REQUIRES_APPROVAL	Routes runtime request to human oversight.
Relationship history/effective dates	Prevents stale or future relationships from authorizing execution.
Step 4 events	Records requested, validated, blocked, approved, executed and failed actions.
Policy Binding/Runtime Engine	Supplies policy resolution, rule evaluation and final policy decision.

Source boundary: Existing GuardianIQ foundations already establish relationship-driven runtime checks, USES_TOOL/USES_DATA_SOURCE/GOVERNED_BY semantics, effective dates, policy evaluation and agent/tool audit events. The detailed autonomy, operation-level tool permission, data-field enforcement, authorization-token and gateway mechanics below are engineering elaborations required to make ENFORCE build-ready.

4. Target Enforcement Architecture





5. Governed Runtime Request Model

```

{
  "request_id": "REQ-20260810-001",
  "correlation_id": "CORR-RFQ-1044",
  "tenant_id": "<derived-server-side>",
  "actor": {"type": "SERVICE", "id": "agent-runtime"},
  "agent": {"id": "AGT-QUOTE-001", "version": "7"},
  "workflow": {"id": "WF-RFQ-001", "run_id": "RUN-7781"},
  "model": {"id": "MOD-LLM-001", "version": "2026-07"},
  "operation": "ERP_PRICE_UPDATE",
  "tool": {"id": "TOOL-SAP-PRICE", "capability": "UPDATE_PRICE", "access_mode": "WRITE"},
  "data_requests": [
    {"source_id": "DS-CUSTOMER", "purpose": "QUOTE_PREPARATION",
     "fields": ["customer_id", "country", "credit_status"]}
  ],
  "facts": {"quote_value": 750000, "risk_score": 86, "environment": "PRODUCTION"},
  "idempotency_key": "RUN-7781:ERP_PRICE_UPDATE:1"
}

```

6. Agent Identity and Runtime Principal

Requirement	Implementation
Stable identity	Every agent has immutable agent_id; versions/configurations are separate.
Runtime principal	Service/workload identity authenticates the executing runtime; it cannot simply claim an arbitrary agent_id.
Agent status	Only ACTIVE agents execute; SUSPENDED/RETIRED/QUARANTINED fail closed.
Owner	Business/technical owner must be resolvable for governed production agents.
Environment	DEV/TEST/PROD authorization is explicit.
Version	Runtime request carries agent version/config hash for audit.
Delegation	Agent-to-agent delegation requires explicit active relationship/permission and bounded scope.

7. Agent Autonomy Boundary Model

Autonomy level	Meaning	Default enforcement
RECOMMEND_ONLY	Agent may produce recommendation/output only	No external WRITE/EXECUTE.
HUMAN_APPROVAL_REQUIRED	Agent may prepare action but human must authorize	All material WRITE/EXECUTE routed to approval.
LIMITED_AUTONOMOUS	Agent executes pre-approved bounded operations	Limits, tools, data, risk and value enforced.
AUTONOMOUS	Broader execution within explicit controls	Still subject to policies, permissions and kill switch.
QUARANTINED	Agent may not perform governed runtime actions	DENY.

7.1 Boundary attributes

Attribute	Example
allowed_operations	SEARCH_PRODUCT, CREATE_DRAFT_QUOTE
blocked_operations	DELETE_CUSTOMER, CHANGE_BANK_ACCOUNT
allowed_tool_ids	TOOL-CRM-READ, TOOL-SAP-QUOTE
allowed_model_ids	MOD-LLM-001
allowed_data_classes	PUBLIC, INTERNAL, CONFIDENTIAL
max_transaction_value	500000
max_records_per_action	100

max_actions_per_hour	60
allowed_environments	TEST, PRODUCTION
approval_thresholds	{"quote_value":500000}
valid_from / valid_to	Effective period
kill_switch	true/false

Precedence: Explicit blocked operation, kill switch, inactive agent, missing relationship or prohibited data classification always dominates an otherwise permissive autonomy level.

8. Model Invocation Boundary

Check	Example
USES_MODEL relationship	AGT-001 may use MOD-LLM-001
Model status/version	Approved ACTIVE version only
Risk tier compatibility	High-risk use case cannot use unapproved experimental model
Provider restriction	RESTRICTED data cannot be sent to prohibited external provider
Region/residency	Model endpoint region must satisfy data policy
Capability	Model supports requested capability
Prompt/data obligation	Mask/tokenize before provider call when required
Fallback model	Fallback must independently pass all checks; never silently bypass governance

9. Tool Registry and Capability Model

Entity	Fields
Tool	tool_id, name, type, owner, environment, status, connector/adaptor, risk_level
Capability	capability_code, operation_type, description, risk_level, parameter_schema
Tool credential reference	vault_reference only; no secret value in GuardianIQ
Endpoint	logical endpoint/environment; no unrestricted arbitrary URL from agent
Capability constraint	allowed parameter ranges, required fields, sensitive parameters
Health	availability, last validated, connector version

9.1 Example capabilities

Tool	Capability	Mode	Risk
SAP Pricing	READ_PRICE	READ	LOW
SAP Pricing	UPDATE_PRICE	WRITE	HIGH
CRM	READ_CUSTOMER	READ	MEDIUM
CRM	UPDATE_CUSTOMER	WRITE	HIGH
Email	CREATE_DRAFT	WRITE	MEDIUM
Email	SEND_EMAIL	EXECUTE	HIGH

10. Tool Permission Model

Permission field	Purpose
agent_id	Who receives entitlement
tool_id	Which registered tool
capability_code	Exact operation
access_mode	READ/WRITE/EXECUTE
environment	DEV/TEST/PROD
parameter_constraints_json	Allowed ranges/values
requires_approval	Base permission can itself require approval
approval_role/group	Who authorizes
max_value/max_records	Operational limit
effective_from/to	Temporal boundary
status	ACTIVE/SUSPENDED/REVOKED/EXPIRED

Tool permission decision:

1. Is USES_TOOL relationship active?
2. Does a matching capability permission exist?
3. Does requested access_mode match?
4. Is environment allowed?
5. Are parameters within constraints?
6. Are value/volume/rate limits satisfied?
7. Does policy add DENY/approval/obligations?
8. If all pass, issue short-lived authorization for this exact request.

11. Tool Execution Gateway

- All governed WRITE/EXECUTE calls must pass through a server-side gateway or controlled adapter.
- Agent cannot submit arbitrary endpoint, credential or HTTP method; it requests a registered capability.
- Gateway resolves connector and credentials internally after authorization.
- Parameters are validated against the registered capability schema and permission constraints.
- Authorization is checked immediately before target invocation.
- Result is normalized and sensitive response fields are filtered before returning to the agent.
- Connector execution timeout, retry and idempotency are capability-specific.

```

async def execute_tool(req, principal):
    ctx = await governance_context.build(req, principal)
    rel = await relationship_guard.validate(ctx)
    boundary = await boundary_service.resolve(ctx, rel)
    policy = await policy_engine.evaluate(ctx, boundary)

    decision = decision_combiner.combine(rel, boundary, policy)
    if decision.is_denied:
        return await enforcement.block(ctx, decision)

    if decision.requires_approval:
        return await approval_service.create_pending(ctx, decision)

    authz = await authorization_service.issue(ctx, decision)
    await authorization_service.verify_immediately_before_execution(authz, req)

    safe_params = obligation_service.transform_request(req.parameters, decision.obligations)
    result = await tool_adapter.execute_registered_capability(req.tool_id, req.capability, safe_params)
    safe_result = obligation_service.transform_response(result, decision.obligations)
    await enforcement.record_outcome(ctx, decision, result)
    return safe_result

```

12. Data Source Registry and Classification

Field	Purpose
data_source_id	Stable registered source
source_type	DB/API/file/lake/warehouse/document store
owner/steward	Accountability
classification	PUBLIC/INTERNAL/CONFIDENTIAL/RESTRICTED
contains_pii	Boolean + categories where applicable
region/residency	Storage/processing region
allowed_purposes	Business purposes
schema_reference	Registered schema/data contract
field_classification	Per-field override
status	ACTIVE/SUSPENDED/RETIRED

13. Data Permission Model

Dimension	Examples
Access operation	READ, QUERY, WRITE, EXPORT
Purpose	QUOTE_PREPARATION, SUPPORT, FRAUD_REVIEW
Field scope	customer_id allowed; bank_account denied
Row scope	region=INDIA; assigned_accounts_only

Classification ceiling	up to CONFIDENTIAL
PII category	contact permitted; government_id denied
Transformation	MASK, TOKENIZE, REDACT, HASH, AGGREGATE
Model/provider constraint	Only approved internal model for RESTRICTED
Export	No bulk export; max 100 records
Retention	Do not persist raw output; ephemeral processing
Effective period	Start/end
Approval	Required for sensitive export/write

14. Data Access Enforcement and Transformation

1. Resolve active USES_DATA_SOURCE relationship.
2. Resolve agent data permission for requested source, operation and purpose.
3. Validate source and field classification.
4. Apply row/record scope.
5. Apply field allow/deny rules.
6. Resolve applicable policy and model/provider restrictions.
7. Apply required masking/tokenization/redaction before model/tool exposure.
8. Enforce maximum record count/export limits.
9. Return only transformed authorized data.
10. Record data-access decision and evidence without unnecessarily copying sensitive payloads into events.

Critical rule: GuardianIQ should enforce the smallest necessary data exposure. A permitted data source does not automatically permit every table, row, field, purpose, export or downstream model.

15. Cross-Boundary Decision Model

Layer	Can ALLOW create entitlement?	Can DENY/block?
Agent status/autonomy	No	Yes
Relationship guard	No	Yes
Tool permission	No	Yes
Data permission	No	Yes
Model boundary	No	Yes
Policy engine	No - it governs existing entitlement	Yes / approval / obligations
Approval	No - only authorizes the evaluated request	Yes by rejection/expiry
Exception	Only within explicitly approved exception scope	N/A

Combination rule: Final ALLOW requires every mandatory boundary layer to pass. Any authoritative DENY blocks. REQUIRE_APPROVAL blocks immediate execution until valid approval. Obligations are cumulative unless contradictory, in which case the stricter obligation wins or the request is rejected as a configuration conflict.

16. Relationship Resolution

Required runtime relationship examples:

```
AGENT --PARTICIPATES_IN--> WORKFLOW
AGENT --USES_MODEL-----> MODEL
AGENT --USES_TOOL-----> TOOL
AGENT --USES_DATA_SOURCE-> DATA_SOURCE
AGENT/WORKFLOW/TOOL/DATA --GOVERNED_BY--> POLICY
```

Validation:

- same tenant
- ACTIVE
- effective now
- required attributes present
- source/target lifecycle permits execution
- relationship constraints satisfied
- history preserved for audit

17. Policy Engine Integration

- Boundary checks produce normalized facts for the policy engine rather than duplicating policy logic.
- Policy resolver receives agent, model, tool, data, workflow, operation, risk, environment and classification context.
- Policy decision is combined with hard permissions; policy cannot override missing entitlement.
- ALLOW_WITH_OBLIGATIONS may require masking, evidence, rate limit, logging, approval for a sub-step or human confirmation.
- Every authoritative evaluation records exact policy version and binding set.

18. Approval, Delegation and Exception Integration

Control	Implementation
Approval	Request contains immutable context hash, policy/rule, agent, tool/data operation, parameters summary and expiry.
Delegation	Human or agent delegation must be explicit, scoped, effective-dated and cannot exceed delegator authority.
Exception	Time-bounded, target/operation-specific, approved and auditable; never generic bypass.
Emergency override	Separate restricted permission, mandatory rationale, short expiry, alerts and post-review.
Reapproval	Required if critical request facts, policy version, target, parameters or risk materially change.

19. Runtime Authorization Token and TOCTOU Protection

```

Authorization token claims (logical):
authorization_id
tenant_id
request_id
agent_id + agent_version
workflow_run_id
model_id/version
tool_id/capability OR data_source/operation
critical_parameter_hash
resolved_relationship_hash
resolved_policy_hash
approval_id (if any)
issued_at / expires_at
single_use / idempotency_key
signature/hash

```

- Token is short-lived and preferably single-use for high-risk WRITE/EXECUTE.
- Adapter verifies token immediately before target call.
- Material parameter change invalidates authorization.
- Expired approval or changed policy/binding can force re-evaluation according to configured risk rule.
- Do not expose a reusable broad bearer token to the agent.

20. Database Physical Design

agent_runtime_boundaries

Column	Type	Purpose
id	UUID PK	Boundary
tenant_id	UUID	Tenant
agent_id	UUID/VARCHAR	Agent
autonomy_level	VARCHAR	Mode
allowed_operations	JSONB	Allow list
blocked_operations	JSONB	Deny list
allowed_model_ids	JSONB	Models
allowed_data_classes	JSONB	Ceiling

max_transaction_value	NUMERIC	Limit
max_records_per_action	INT	Limit
max_actions_per_hour	INT	Rate
allowed_environments	JSONB	Env
kill_switch	BOOLEAN	Emergency stop
effective_from	TIMESTAMPTZ	Start
effective_to	TIMESTAMPTZ	End
status	VARCHAR	Lifecycle

tool_capabilities

Column	Type	Purpose
id	UUID PK	Capability
tenant_id	UUID	Tenant
tool_id	UUID/VARCHAR	Tool
capability_code	VARCHAR	Operation
access_mode	VARCHAR	READ/WRITE/EXECUTE
risk_level	VARCHAR	Risk
parameter_schema	JSONB	Validation
response_schema	JSONB	Optional output contract
status	VARCHAR	Lifecycle

agent_tool_permissions

Column	Type	Purpose
id	UUID PK	Permission
tenant_id	UUID	Tenant
agent_id	UUID/VARCHAR	Agent
tool_id	UUID/VARCHAR	Tool
capability_code	VARCHAR	Capability
access_mode	VARCHAR	Mode
environment	VARCHAR	Env
parameter_constraints	JSONB	Limits
requires_approval	BOOLEAN	Human control
approval_group_id	UUID NULL	Group
max_value	NUMERIC NULL	Value limit
max_records	INT NULL	Volume
effective_from/to	TIMESTAMPTZ	Validity
status	VARCHAR	Lifecycle

data_source_fields

Column	Type	Purpose
id	UUID PK	Field
tenant_id	UUID	Tenant
data_source_id	UUID/VARCHAR	Source
field_path	VARCHAR	Schema path
classification	VARCHAR	Sensitivity
pii_category	VARCHAR NULL	PII type
default_masking	VARCHAR NULL	Transform
status	VARCHAR	Lifecycle

agent_data_permissions

Column	Type	Purpose
id	UUID PK	Permission
tenant_id	UUID	Tenant
agent_id	UUID/VARCHAR	Agent
data_source_id	UUID/VARCHAR	Source
operation	VARCHAR	READ/QUERY/WRITE/EXPORT
purpose_code	VARCHAR	Purpose
allowed_fields	JSONB	Allow
denied_fields	JSONB	Deny
row_scope	JSONB	Predicate model
classification_ceiling	VARCHAR	Max class
transformations	JSONB	Mask/tokenize

max_records	INT	Limit
requires_approval	BOOLEAN	Control
effective_from/to	TIMESTAMPTZ	Validity
status	VARCHAR	Lifecycle

runtime_authorizations

Column	Type	Purpose
id	UUID PK	Authorization
tenant_id	UUID	Tenant
request_id	VARCHAR	Request
correlation_id	VARCHAR	Flow
agent_id	VARCHAR	Agent
target_type/id	VARCHAR	Target
operation	VARCHAR	Operation
context_hash	VARCHAR	Exact request
relationship_hash	VARCHAR	Relationships
policy_hash	VARCHAR	Policies
approval_id	UUID NULL	Approval
decision	VARCHAR	Decision
issued_at	TIMESTAMPTZ	Issued
expires_at	TIMESTAMPTZ	Expiry
consumed_at	TIMESTAMPTZ NULL	Single use
status	VARCHAR	Lifecycle

runtime_enforcement_log

Column	Type	Purpose
id	UUID PK	Log
tenant_id	UUID	Tenant
request_id	VARCHAR	Request
correlation_id	VARCHAR	Flow
agent_id	VARCHAR	Agent
model_id	VARCHAR NULL	Model
tool_id	VARCHAR NULL	Tool
data_source_ids	JSONB	Data
operation	VARCHAR	Operation
decision	VARCHAR	Result
reason_code	VARCHAR	Machine reason
obligations	JSONB	Controls
authorization_id	UUID NULL	Authz
execution_reference	VARCHAR NULL	Target result ref
created_at	TIMESTAMPTZ	Time

20.1 Indexes and constraints

```

CREATE INDEX idx_agent_boundary_active
  ON agent_runtime_boundaries(tenant_id, agent_id, status, effective_from, effective_to);

CREATE UNIQUE INDEX uq_tool_capability
  ON tool_capabilities(tenant_id, tool_id, capability_code);

CREATE INDEX idx_agent_tool_permission
  ON agent_tool_permissions(tenant_id, agent_id, tool_id, capability_code, status);

CREATE INDEX idx_agent_data_permission
  ON agent_data_permissions(tenant_id, agent_id, data_source_id, operation, purpose_code, status);

CREATE UNIQUE INDEX uq_runtime_request_auth
  ON runtime_authorizations(tenant_id, request_id);

CREATE INDEX idx_enforcement_corr
  ON runtime_enforcement_log(tenant_id, correlation_id, created_at DESC);

```

21. Backend Module Architecture

```

backend/modules/enforcement/
  schemas.py
  enums.py
  gateway.py
  context_builder.py
  decision_combiner.py
  authorization_service.py
  obligation_service.py
  enforcement_repository.py
  router.py

backend/modules/agent_boundary/
  models.py
  repository.py
  service.py
  resolver.py
  rate_limit.py

backend/modules/tool_governance/
  capability_service.py
  permission_service.py
  tool_guard.py
  adapters/
  parameter_validator.py

backend/modules/data_governance/
  classification_service.py
  permission_service.py
  data_guard.py
  row_scope.py
  field_filter.py
  transformer.py

backend/modules/model_governance/
  model_guard.py
  provider_guard.py

backend/modules/relationships/ # existing architecture
backend/modules/policy_engine/ # prior specification
backend/modules/approvals/
backend/modules/events/      # Step 4

```

22. Backend Service Specifications

Service	Responsibility
RuntimePrincipalService	Authenticate workload and bind it to registered agent/runtime.
AgentBoundaryResolver	Return effective autonomy and hard limits.
ModelGuard	Validate model relationship/status/provider/data compatibility.
ToolGuard	Validate USES_TOOL + capability permission + constraints.
DataGuard	Validate USES_DATA_SOURCE + purpose/field/row/classification rules.
RelationshipGuard	Resolve active effective relationships.
PolicyEngineAdapter	Call authoritative policy resolution/evaluation.
DecisionCombiner	Combine hard boundary + policy + approval/exception result.
AuthorizationService	Issue/verify short-lived exact-request authorization.
ObligationService	Apply mask/tokenize/redact/rate/evidence obligations.
EnforcementRepository	Persist runtime decision/evidence.
EventPublisher	Publish append-only governance events through outbox.

23. Runtime Interceptor Specifications

Interceptor	When invoked	Failure behavior
AgentActionInterceptor	Before any governed agent action	Block if agent boundary invalid.
ModelInvocationInterceptor	Before prompt/data leaves platform	Block or transform before provider call.
ToolAccessInterceptor	Before registered tool capability execution	Block before credential/connector invocation.

DataAccessInterceptor	Before query/read/export or before data passed downstream	Filter/mask/block.
WorkflowRunInterceptor	Before workflow activation/run step	Block if mandatory governance/boundaries invalid.
ActionExecutionInterceptor	Immediately before target mutation	Reverify authorization/context hash.

24. REST API Contracts

Method	Endpoint	Purpose
GET	/api/v1/agents/{id}/boundary	Current effective boundary
PUT	/api/v1/agents/{id}/boundary	Create/update draft/effective boundary
POST	/api/v1/agents/{id}/kill-switch	Emergency stop
GET	/api/v1/tools/{id}/capabilities	Capabilities
POST	/api/v1/tools/{id}/capabilities	Register capability
GET	/api/v1/agents/{id}/tool-permissions	Agent tool entitlements
POST	/api/v1/agent-tool-permissions	Grant governed permission
POST	/api/v1/agent-tool-permissions/{id}/revoke	Revoke
GET	/api/v1/data-sources/{id}/fields	Field classification
POST	/api/v1/agent-data-permissions	Grant data permission
GET	/api/v1/agents/{id}/data-permissions	Permissions
POST	/api/v1/enforcement/evaluate	Internal pre-execution decision
POST	/api/v1/enforcement/authorize	Issue exact-request authorization
POST	/api/v1/enforcement/execute/tool	Controlled tool execution
GET	/api/v1/enforcement/history	Runtime decisions
POST	/api/v1/enforcement/simulate	Non-authoritative simulation

25. Frontend Information Architecture

Agent Detail

- └ Overview
- └ Models
- └ Tool Access
- └ Data Access
- └ Policies
- └ Autonomy & Limits
- └ Enforcement History

Tool Detail

- └ Capabilities
- └ Agent Permissions
- └ Policies
- └ Access History

Data Source Detail

- └ Classification
- └ Field Classification
- └ Agent Permissions
- └ Policies
- └ Access History

Governance > Enforcement

- └ Boundary Management
- └ Permission Matrix
- └ Simulation
- └ Blocked Actions
- └ Runtime Decision Detail

26. Frontend Screen Specifications

Screen	Must show/do
Agent Autonomy & Limits	Autonomy level, status, kill switch, allowed/blocked operations, models, data classes, value/volume/rate limits, effective dates.
Agent Tool Access	Tool → capability → mode → environment → constraints → approval → status; grant/revoke drawer.

Agent Data Access	Source → purpose → operation → fields → classification ceiling → transforms → max records → approval.
Tool Capability Manager	Register capability, mode, risk, parameter schema, connector status.
Data Classification	Source and field classifications, PII categories, default transforms.
Permission Matrix	Agent x tool/data compact view with filters and gaps.
Enforcement Simulation	Select agent/workflow/model/tool/data/operation + facts; preview relationship/boundary/policy decision.
Blocked Actions	Reason, agent, target, operation, policy, permission failure, owner, timestamp, correlation.
Runtime Decision Detail	Layer-by-layer decision trace, policy evaluation, approval, obligations, authorization, execution outcome.
Enforcement History	Allow/block/approval timeline; filters by agent/tool/data/policy/decision.

27. Frontend Component Behavior

- Never expose raw credentials or secret references beyond safe metadata.
- Permission grant UI validates capability and data-source existence before submit.
- High-risk WRITE/EXECUTE grants require reason and optional approval workflow.
- Kill switch requires confirmation, reason and elevated permission; status change is immediately visible.
- Applicable policy and hard permission are shown separately so users understand why an action is blocked.
- Simulation is labelled NON-AUTHORITATIVE and cannot call target systems.
- Blocked action detail should show a concise remediation hint such as 'Missing WRITE capability' or 'RESTRICTED data cannot be sent to provider X'.
- All enforcement administration mutations use backend authorization; hiding buttons is not security.

28. Governance Events and Audit Evidence

Event	Trigger
AGENT_ACTION_REQUESTED	Before governed action
AGENT_ACTION_VALIDATED	Boundary checks pass
AGENT_ACTION_BLOCKED	Any hard/policy boundary blocks
TOOL_ACCESS_ATTEMPTED	Before tool capability
TOOL_ACCESS_DENIED	Tool relationship/permission/policy denies
DATA_ACCESS_REQUESTED	Implementation extension: before governed data access
DATA_ACCESS_DENIED	Implementation extension: denied data request
DATA_TRANSFORMATION_APPLIED	Implementation extension: masking/tokenization/redaction
MODEL_INVOCATION_REQUESTED	Implementation extension: before model provider call
MODEL_INVOCATION_BLOCKED	Implementation extension: model/data/provider incompatibility
POLICY_EVALUATED / POLICY_TRIGGERED	Policy engine
APPROVAL_REQUESTED / GRANTED / REJECTED	Human control
ACTION_EXECUTED / ACTION_FAILED	Outcome
AGENT_KILL_SWITCH_ACTIVATED	Implementation extension: emergency boundary

Event catalogue change: Events marked as implementation extensions should be added to the Step 4 event schema registry and versioned before production use.

29. Security Architecture

Threat/control	Required design
Agent spoofing	Authenticated workload identity mapped to registered agent/runtime.
Privilege escalation	Agent cannot request a capability not explicitly granted.
Cross-tenant access	Tenant derived server-side; repository filters and cache isolation.
Prompt/tool injection	Agent output cannot directly define endpoint/method/credential; only registered capability IDs.
Sensitive data exfiltration	Classification + field filters + model/provider restrictions + export limits.
Approval replay	Short-lived single-use context-bound authorization.
Stale permission	Effective dates + cache invalidation + immediate recheck for critical execution.

Audit tampering	Append-only events, hashes where configured, restricted event DB role.
Emergency access abuse	Separate role, short expiry, reason, alert, post-review.

30. Secrets and Credential Handling

- GuardianIQ stores only credential references, never plaintext secrets.
- Tool adapter resolves secret from enterprise vault at execution time after authorization.
- Agent never receives reusable database/API credentials.
- Use least-privilege service accounts per connector/environment where feasible.
- Rotate credentials independently of GuardianIQ permissions.
- Secret access itself should be logged by the vault/platform and correlated where possible.
- Do not place tokens, API keys or raw authorization headers in governance events.

31. Performance, Caching and Resilience

Area	Target/design
Common cached evaluation path	p95 target < 200 ms excluding target system/approval
Boundary cache	tenant+agent+version; invalidate on boundary/status/kill switch change
Tool permission cache	tenant+agent+tool+capability+environment
Data permission cache	tenant+agent+source+operation+purpose
Relationship cache	Only effective active relationships; critical changes invalidate immediately
Policy cache	Reuse immutable active version/hash from policy engine
Gateway scale	Stateless replicas; DB/cache external
Cache failure	Fallback to authoritative DB; never bypass
Policy/relationship dependency timeout	Critical WRITE/EXECUTE fail closed
Target retries	Capability-specific and idempotency-aware

32. Error Handling and Fail-Closed Rules

Condition	Result
Agent inactive/quarantined/kill switch	DENY
Missing required relationship	DENY
No matching tool capability permission	DENY
Requested parameter outside constraint	DENY or REQUIRE_APPROVAL if explicitly configured
Data classification exceeds ceiling	DENY
Denied field requested	Remove only if safe and policy permits partial response; otherwise DENY
Model/provider incompatible with data	DENY before provider call
Mandatory policy unavailable/config error	DENY critical operation + alert
Approval missing/expired	PENDING/BLOCK
Context hash changed	Invalidate authorization and re-evaluate
Connector unavailable after authorization	ACTION_FAILED; do not convert to policy denial
Event dispatch transient failure	Persist outbox; do not lose enforcement record

33. Observability and Operations

Metric/alert	Why
agent_actions_total by decision	Overall enforcement activity
blocked_actions by reason	Find permission/policy/configuration issues
tool_access_denied by capability	Tool governance
data_access_denied by classification/source	Data governance
model_invocation_blocked by provider	Provider/data compatibility
kill_switch_active	Critical operational status
enforcement_latency p95/p99	User/runtime impact
authorization_context_mismatch	Potential TOCTOU/tampering
expired_permission/binding hits	Configuration hygiene
approval backlog from runtime	Human bottleneck
exception/override usage	Governance risk

cache hit ratio / dependency timeout	Platform health
--------------------------------------	-----------------

34. Detailed End-to-End Scenarios

A. Agent sends a quote email

Context	Control evaluation	Expected outcome
Agent asks Email SEND_EMAIL.	Agent active; SEND_EMAIL capability exists but autonomy=HUMAN_APPROVAL_REQUIRED.	Create approval. After approval and unchanged context, issue authorization and execute; audit full chain.

B. Agent tries SAP price update without WRITE

Context	Control evaluation	Expected outcome
USES_TOOL exists READ_ONLY; request UPDATE_PRICE/WRITE.	Relationship exists but capability permission does not.	DENY before SAP credential/endpoint invocation; TOOL_ACCESS_DENIED + AGENT_ACTION_BLOCKED.

C. Restricted customer data to external model

Context	Control evaluation	Expected outcome
Agent may use source and model independently.	Data classification=RESTRICTED; provider policy prohibits external processing.	DENY before prompt leaves platform; no restricted data transmitted.

D. Allowed data with masking

Context	Control evaluation	Expected outcome
CONFIDENTIAL contact data needed for analysis.	Permission allows source/purpose but requires MASK email/phone.	Transform data, then model invocation; record obligation/evidence.

E. High-value transaction

Context	Control evaluation	Expected outcome
Agent LIMITED_AUTONOMOUS max value=500k; request=750k.	Hard autonomy limit exceeded and policy requires approval.	Hard limit cannot be expanded by simple policy ALLOW; route according to configured approval/exception design or DENY.

F. Kill switch

Context	Control evaluation	Expected outcome
Security admin activates agent kill switch.	Any subsequent action.	Immediate DENY even if cached permissions/policies previously allowed.

G. Permission revoked during pending approval

Context	Control evaluation	Expected outcome
WRITE permission revoked after approval request.	Execution attempted later.	Revalidation fails; approval cannot resurrect revoked entitlement.

H. Agent delegates to another agent

Context	Control evaluation	Expected outcome
AGT-A asks AGT-B to execute tool.	Delegation relationship/scope missing.	DENY; agent-to-agent handoff is governed.

I. Bulk export

Context	Control evaluation	Expected outcome
---------	--------------------	------------------

Agent requests 10,000 records; permission max=100.	Limit exceeded.	DENY or require governed batch/approval path; never silently export.
--	-----------------	--

J. Model fallback

Context	Control evaluation	Expected outcome
Primary model unavailable; runtime selects fallback.	Fallback model has no USES_MODEL approval for RESTRICTED data.	Block fallback; do not bypass model governance.

35. Testing Strategy

Test type	Coverage
Unit	Autonomy precedence, capability matching, field filters, row scope, transforms, context hashing, decision combination
Repository	Tenant isolation, effective dates, revoked permissions, indexes
API	Grant/revoke, kill switch, simulation, enforcement, history
Integration	Agent→relationship→boundary→policy→approval→adapter→events
Connector	Parameters, idempotency, credential resolution, normalized errors
Data security	Field denial, masking, classification ceiling, export limit, provider restriction
Security	Spoofed agent, cross-tenant, replayed authorization, privilege escalation
Failure	Cache/DB/policy timeout, connector failure, event outbox failure
Performance	High-frequency read checks and controlled concurrent execution
Audit	Full correlation and decision reconstruction

36. Acceptance Test Catalogue

ID	Test	Expected
ENF-001	Inactive agent attempts action	Denied before downstream call
ENF-002	Kill switch active	All governed actions denied
ENF-003	RECOMMEND_ONLY agent attempts WRITE	Denied
ENF-004	Agent uses unapproved model	Denied
ENF-005	Model version not active	Denied
ENF-006	USES_TOOL missing	Denied
ENF-007	Tool READ permission attempts WRITE	Denied
ENF-008	Capability parameter above max	Denied/approval per configured rule
ENF-009	Correct tool permission	Passes to policy layer
ENF-010	USES_DATA_SOURCE missing	Denied
ENF-011	Wrong purpose	Denied
ENF-012	Denied field requested	Filtered/denied as configured
ENF-013	Classification above ceiling	Denied
ENF-014	Masking obligation	Data transformed before exposure
ENF-015	Restricted data + prohibited provider	Blocked before model call
ENF-016	Record export exceeds limit	Denied
ENF-017	Policy DENY after permissions pass	Denied
ENF-018	Policy requires approval	No execution until approval
ENF-019	Approval granted unchanged context	Execution allowed
ENF-020	Request changed after approval	Authorization invalid; re-evaluate
ENF-021	Permission revoked after approval	Execution denied
ENF-022	Authorization replay	Rejected if single-use
ENF-023	Cross-tenant permission query	Rejected/no data
ENF-024	Spoofed agent id	Rejected
ENF-025	Cache unavailable	Authoritative fallback, same decision
ENF-026	Critical policy engine timeout	Fail closed
ENF-027	Connector failure after allow	ACTION_FAILED, not policy denial
ENF-028	Agent-to-agent delegation missing	Denied
ENF-029	Valid delegation within scope	Allowed subject to all downstream checks
ENF-030	Event correlation	Request, policy, approval, execution share correlation_id
ENF-031	Audit view	Shows relationship/boundary/policy/approval/outcome
ENF-032	Simulation	No target side effect
ENF-033	Emergency override	Restricted role + expiry + audit
ENF-034	Expired permission	Denied
ENF-035	Fallback model not authorized	Blocked

37. Implementation Sequence

11. Freeze enums, runtime request schema and enforcement reason codes.
12. Create additive PostgreSQL migrations and indexes.
13. Implement agent boundary repository/service and kill switch.
14. Implement tool capability registry and agent tool permission service.
15. Implement data field classification and agent data permission service.
16. Integrate existing relationship resolver.
17. Implement model guard/provider compatibility.
18. Implement governance context builder.
19. Implement hard-boundary guards and deterministic decision combiner.
20. Integrate Policy Binding/Resolution/Runtime Enforcement engine.
21. Integrate approval/exception/delegation.
22. Implement runtime authorization/context hash.
23. Implement controlled tool/model/data interceptors/adapters.
24. Extend Step 4 event catalogue and outbox publishing.
25. Build frontend Agent Autonomy, Tool Access and Data Access tabs.
26. Build Tool Capability and Data Classification admin screens.
27. Build Enforcement Simulation, Blocked Actions and Runtime Decision Detail.
28. Add metrics, alerts and support diagnostics.
29. Run observe-only pilot and compare expected decisions.
30. Enable enforcement progressively: READ-sensitive → WRITE → EXECUTE/high-risk.
31. Complete acceptance catalogue, UAT and architecture sign-off.

38. Resource / Workstream Breakdown

Workstream	Primary deliverables	Suggested profile
DB & backend core	Tables, repositories, services, indexes	Backend/DB
Agent boundary	Autonomy, limits, kill switch, delegation hooks	Backend
Tool governance	Capabilities, permissions, gateway adapters	Backend/Integration
Data governance	Classification, field/row scope, transforms	Backend/Data
Policy/approval integration	Decision combination, approval/exception	Backend
Frontend	Agent/tool/data governance UX, simulation/history	Frontend
QA	35 acceptance tests, security/failure/performance	QA
DevOps/Security	Service identity, vault integration, metrics, rollout	DevOps/Security

39. Deployment and Migration

32. Deploy schema and APIs behind feature flags.
33. Seed capability/access/classification reference values.
34. Populate agent boundaries from existing registry defaults; do not infer broad permissions.
35. Map existing USES_TOOL/USES_DATA_SOURCE relationships to permissions only after owner review.
36. Run enforcement in OBSERVE_ONLY for selected agents and compare expected vs actual.
37. Enable kill switch and deny on inactive agents first.
38. Enable tool WRITE/EXECUTE enforcement for pilot.
39. Enable sensitive data and model/provider enforcement.
40. Enable approval/context-bound execution.
41. Expand by agent/workflow/business unit with operational monitoring.

40. Definition of Done

- Every production agent has an explicit effective autonomy boundary.

- Every governed tool operation maps to a registered capability.
- WRITE/EXECUTE requires active relationship + matching permission + policy decision + approval when required.
- Sensitive data access is source/purpose/field/classification constrained.
- Restricted data cannot reach an unauthorized model/provider.
- All controlled target calls pass through a server-side interceptor/gateway.
- Short-lived exact-request authorization protects high-risk execution.
- Kill switch blocks immediately.
- Runtime events/evidence reconstruct the complete decision.
- Frontend provides administration, simulation and enforcement visibility.
- Tenant isolation, RBAC/ABAC and secret handling pass security tests.
- Acceptance tests pass and pilot enforcement is demonstrated end to end.

41. Handover Checklist

Artifact	Status required
DB migrations + rollback	Complete
OpenAPI contracts	Complete
Runtime request schema	Frozen v1
Agent boundary rules	Documented/tested
Tool capability catalogue	Seeded/pilot ready
Data classification model	Approved
Policy/approval integration tests	Passing
Event schema extensions	Registered
Frontend screens	UAT ready
Security test evidence	Passing
Observe-only comparison report	Reviewed
Production runbook	Complete
Kill-switch runbook	Complete
Monitoring/alerts	Configured
Architecture/Product/QA sign-off	Complete

Appendix A. Core Enums

Enum	Values
AutonomyLevel	RECOMMEND_ONLY, HUMAN_APPROVAL_REQUIRED, LIMITED_AUTONOMOUS, AUTONOMOUS, QUARANTINED
AccessMode	READ, WRITE, EXECUTE
DataOperation	READ, QUERY, WRITE, EXPORT
Classification	PUBLIC, INTERNAL, CONFIDENTIAL, RESTRICTED
BoundaryStatus	DRAFT, ACTIVE, SUSPENDED, EXPIRED, REVOKED
Decision	ALLOW, ALLOW_WITH_OBLIGATIONS, DENY, REQUIRE_APPROVAL, ESCALATE
Environment	DEV, TEST, STAGING, PRODUCTION
Transformation	MASK, TOKENIZE, REDACT, HASH, AGGREGATE
AuthorizationStatus	ISSUED, CONSUMED, EXPIRED, REVOKED
EnforcementMode	OBSERVE_ONLY, ENFORCE

Appendix B. Example Permission Records

```
Agent tool permission
{
  "agent_id": "AGT-QUOTE-001",
  "tool_id": "TOOL-SAP-PRICE",
  "capability_code": "UPDATE_PRICE",
  "access_mode": "WRITE",
  "environment": "PRODUCTION",
  "parameter_constraints": {"currency": ["INR", "USD"], "max_value": 500000},
  "requires_approval": true,
  "status": "ACTIVE"
}
```

```
}

Agent data permission
{
  "agent_id": "AGT-QUOTE-001",
  "data_source_id": "DS-CUSTOMER",
  "operation": "READ",
  "purpose_code": "QUOTE_PREPARATION",
  "allowed_fields": ["customer_id", "country", "credit_status", "email"],
  "denied_fields": ["bank_account", "government_id"],
  "classification_ceiling": "CONFIDENTIAL",
  "transformations": {"email": "MASK"},
  "max_records": 25,
  "status": "ACTIVE"
}
```

Appendix C. Suggested Frontend Structure

```
frontend/src/features/enforcement/
pages/
  EnforcementOverviewPage.jsx
  BlockedActionsPage.jsx
  EnforcementSimulationPage.jsx
  RuntimeDecisionDetailPage.jsx
components/
  AgentBoundaryCard.jsx
  AutonomySelector.jsx
  KillSwitchControl.jsx
  ToolPermissionTable.jsx
  ToolPermissionDrawer.jsx
  DataPermissionTable.jsx
  DataPermissionDrawer.jsx
  CapabilityMatrix.jsx
  DataClassificationBadge.jsx
  DecisionTrace.jsx
  ObligationList.jsx
  AuthorizationStatus.jsx
api/enforcementApi.js
hooks/useAgentBoundary.js
hooks/useToolPermissions.js
hooks/useDataPermissions.js
hooks/useEnforcementHistory.js
```